

中山大學
SUN YAT-SEN UNIVERSITY

Assignment 1 实验报告

MDP

姓名　　：钟泓瑨

学号　　：23336338

教学班号　：202610271

专业　　：计算机科学与技术

院系　　：中山大学

2026 年 10 月 13 日

目录

1 实验要求及问题定义 **2**

2 MDP 建模 **2**

3 算法设计 **3**

3.1 求解思路 3
3.2 算法流程 4
3.3 收敛性与停止准则 5
3.4 策略提取 6
3.5 复杂度分析 6
3.6 实现要点 7

4 实验结果 **8**

5 总结 **8**

1 实验要求及问题定义

- 将给定迷宫建模为马尔可夫决策过程（MDP），使用 Policy Iteration 或 Value Iteration 求解；本实验采用 Value Iteration。
- 状态：用智能体所在格子的坐标 (r, c) 表示，墙体不属于状态空间；`S` 和 `G` 分别为起点和终点。
- 动作：向北、东、南、西（N、E、S、W）四个方向移动，每次一格。
- 状态转移：移动结果确定；若目标位置为墙体或超出边界，则留在原地。
- 奖励与终止：每步奖励为 -1，包括撞墙和进入终点；到达终点后结束，终点价值为 0。
- 目标：在折扣因子 $0 < \gamma < 1$ 下，求使累计折扣回报最大的策略，即以最少步数从起点到达终点。

2 MDP 建模

将迷宫问题建模为马尔可夫决策过程 $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$，五个分量依次为状态空间、动作空间、状态转移概率、奖励函数和折扣因子。

状态空间 $\mathcal{S}$ 由所有可通行格子的坐标 (r, c) 构成，包含起点和终点，终点记为 s_g。行、列编号从 0 开始，分别向下、向右增大。

动作以行、列位移表示，上、右、下、左四个方向对应的动作空间为

$$\mathcal{A} = \{(-1, 0), (0, 1), (1, 0), (0, -1)\}.$$

用 $s + a$ 表示坐标与位移按分量相加的结果。若目标格可通行，则移至该格；撞墙或越界时留在原地。终点为吸收状态，转移函数为

$$f(s, a) = \begin{cases} s_g, & s = s_g, \\ s + a, & s \neq s_g \text{ 且 } s + a \in \mathcal{S}, \\ s, & \text{其他情况.} \end{cases}$$

对应的转移概率为

$$P(s' \mid s, a) = \begin{cases} 1, & s' = f(s, a), \\ 0, & \text{其他情况.} \end{cases}$$

非终点状态的每次动作奖励为 -1，包括撞墙、越界和进入终点。到达终点后回合结束，后续奖励为 0。奖励函数及终点价值为

$$R(s,a)=\begin{cases}-1, & s\neq s_g,\\ 0, & s=s_g,\end{cases}\qquad V(s_g)=0.$$

策略 π 根据当前状态选择动作，其价值函数定义为从状态 s 出发的期望累计折扣回报：

$$V^{\pi}(s)=\mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty}\gamma^{t}R(s_t,a_t)\,\middle|\,s_0=s\right].$$

求解目标是得到最优价值函数 $V^{*}(s)=\max_{\pi}V^{\pi}(s)$ 及对应策略 π^{*}。对于终点可达的状态，若经过 d 步到达终点，回报为 $-\sum_{t=0}^{d-1}\gamma^{t}$。每多走一步，回报减少 γ^{d}，因此最大化回报等价于选择到终点的最短路径。

3 算法设计

3.1 求解思路

由于迷宫的转移与奖励模型在第 2 节中已经完全已知，求解时不需要与环境交互采样，可以直接使用动态规划。记贝尔曼最优算子为 $\mathcal{T}$，它对任意有界价值函数 V 的作用定义为

$$(\mathcal{T}V)(s)=\begin{cases}0, & s=s_g,\\ \max\limits_{a\in\mathcal{A}}\Big\{R(s,a)+\gamma V\big(f(s,a)\big)\Big\}, & s\neq s_g.\end{cases}$$

其中终点被显式地当作吸收态：备份时不再对终点自举，而是直接把价值固定为 $V(s_g)=0$，这与终点的奖励 $R(s_g,\cdot)=0$ 及零后续奖励一致。其余状态的最优性备份只涉及一个后继状态，因此期望算子退化为一次函数求值。

该算子具有两条关键性质：

- **单调性**：若价值函数逐分量满足 $V\le U$，则 $\mathcal{T}V\le\mathcal{T}U$；
- **压缩性**：记最大范数 $\|V\|_{\infty}=\max_{s\in\mathcal{S}}|V(s)|$，则

$$\|\mathcal{T}V-\mathcal{T}U\|_{\infty}\le\gamma\|V-U\|_{\infty}.$$

压缩性的证明只用到不等式 $|\max_a x_a-\max_a y_a|\le\max_a|x_a-y_a|$。对 $s\neq s_g$ 有

$$\big|(\mathcal{T}V)(s)-(\mathcal{T}U)(s)\big|\le\gamma\max_{a\in\mathcal{A}}\big|V\big(f(s,a)\big)-U\big(f(s,a)\big)\big|\le\gamma\|V-U\|_{\infty},$$

而在 $s=s_g$ 处两者之差为 0，故上式对所有状态成立。

由 $\gamma < 1$ 与压缩性，Banach 不动点定理保证 $\mathcal{T}$ 存在唯一不动点 $V^* = \mathcal{T}V^*$，它就是最优价值函数；并且从任意初值出发的迭代 $V_{k+1} = \mathcal{T}V_k$ 都以几何速率 γ^k 收敛到 V^*。基于这一迭代格式的算法即价值迭代（Value Iteration）。

本实验采用价值迭代而不采用策略迭代，原因是两者虽然都收敛到最优策略，但价值迭代只需要一个循环，每一轮对全部状态执行一次最优性备份，每轮开销固定且实现简单；策略迭代的外层轮数通常更少，但每一轮都要把当前策略的策略评估迭代到收敛，因而引入一个内层循环和额外的停止条件。对本问题这样规模很小、转移确定的 MDP，价值迭代在实现与验证成本上更占优势。两种方法的对比见表 1。

表 1: 价值迭代与策略迭代的对比

比较项	价值迭代	策略迭代
迭代结构	单层循环，直接迭代最优价值函数	外层策略改进 + 内层策略评估
每轮开销	$O(\lvert\mathcal{S}\rvert\lvert\mathcal{A}\rvert)$，固定	需要一次（近似）策略评估
收敛保证	压缩映射，误差按 γ^k 几何衰减	有限步内收敛到最优策略
停止条件	只需要 $\max_s \lvert\Delta V(s)\rvert < \varepsilon$	需要策略稳定与评估精度两个条件
本实验	采用	作为替代方案未采用

3.2 算法流程

代码 1 给出同步价值迭代的伪代码。输入为第 2 节建立的模型 $(\mathcal{S}, \mathcal{A}, f, R, \gamma)$ 与容差 ε，输出为最优价值函数与贪心策略。

代码 1: 同步价值迭代伪代码

```
输入：状态集 S，动作集 A，转移 f，奖励 R，折扣 gamma，容差 eps
输出：价值表 V，策略 pi

V[s] <- 0                          # 初始化，终点天然为 0
repeat
    delta <- 0
    for s in S:                    # 同步备份，先写入 V_new
        if s == goal:
            V_new[s] <- 0          # 吸收态，不自举
        else:
            V_new[s] <- max{ R(s,a) + gamma * V[f(s,a)] : a in A }
        delta <- max(delta, |V_new[s] - V[s]|)
    V <- V_new
until delta < eps                  # 停止准则
```

```
15
16 for s in S, s != goal:            # 策略提取
17     pi[s] <- argmax{ R(s,a) + gamma * V[f(s,a)] : a in A }
18 return V, pi
```

流程中的几个要点如下：

1. **初始化**：所有状态（含终点）的价值初始化为 0。取 $V_0 \equiv 0$ 使得 $V_0 \geq V^*$，迭代序列从上方单调下降逼近 V^*，并且 $V_0(s_g) = 0$ 与终点的真实价值一致。
2. **最优性备份**：对每个非终点状态，用当前价值表计算四个动作的 Q 值并取最大，得到该状态的新价值。终点不参与备份，恒为 0。
3. **同步更新**：一轮之内所有状态都读取旧价值表 V_k，结果写入新表 V_{k+1}，整轮结束后再整体替换。这样每次迭代都严格对应于算子 $\mathcal{T}$ 的一次作用，记录的最大更新量 δ_k 就是真实的最大范数差 $\|V_{k+1} - V_k\|_\infty$，可以直接用于误差估计；若改为就地更新（Gauss–Seidel），实践中收敛更快，但 δ_k 不再是算子意义下的干净上界，停止准则的误差推导会变弱。
4. **终止判断**：当一轮中所有状态的价值改变量都不超过 ε 时停止迭代。
5. **策略提取**：收敛后只提取一次策略，避免迭代中期不可靠的价值影响最终结果。

3.3 收敛性与停止准则

记 V_k 为第 k 轮迭代后的价值表，$V_0 \equiv 0$。由于所有非终点状态的单步奖励为 -1、终点价值为 0，价值始终落在区间 $\left[-\frac{1}{1-\gamma}, 0\right]$ 内，因此价值表不会溢出，$\mathcal{T}$ 的良好定义也得到了保证。

在压缩性的基础上，迭代误差有先验界

$$\|V_k - V^*\|_\infty \leq \gamma^k \|V_1 - V_0\|_\infty \leq \gamma^k,$$

其中用到 $\|V_1 - V_0\|_\infty = 1$：因为 $V_1(s) = -1$ $(s \neq s_g)$ 而 $V_0 \equiv 0$。要使误差小于 ε，只需迭代轮数满足 $k \geq \dfrac{\log\big(\varepsilon(1-\gamma)/\gamma\big)}{\log \gamma}$。该估计只利用了最坏情况的初始误差，通常相当保守。

实际实现使用的停止准则是后验的。设第 k 轮的实测更新量为

$$\delta_k = \|V_{k+1} - V_k\|_\infty = \max_{s \in \mathcal{S}} \big|V_{k+1}(s) - V_k(s)\big|,$$

则由压缩性可得标准误差界

$$\|V_{k+1} - V^*\|_\infty \leq \frac{\gamma}{1-\gamma}\, \delta_k.$$

因此取 $\delta_k < \varepsilon$ 作为停止条件时，返回价值的误差不超过 $\dfrac{\gamma\varepsilon}{1-\gamma}$。本实验取 $\gamma = 0.99$、$\varepsilon = 10^{-8}$，于是价值误差上界约为 9.9×10^{-7}，已经远小于迷宫问题的步数尺度，对策略的影响可以忽略。

需要指出的是，**在本题的单位步代价与零初始化下，价值迭代会在有限轮内到达精确不动点**。设 $d(s)$ 为状态 s 到终点的最短步数，$D = \max_s d(s)$。对 d 做归纳可证：任何动作的 Q 值都不超过 -1（因为即时奖励为 -1 且价值非正），而 $V^*(s) = -\sum_{t=0}^{d(s)-1} \gamma^t \le -1$，因此朝终点方向的动作为最优动作；于是经过 k 轮后所有满足 $d(s) \le k$ 的状态已经取到最优值。特别地，当 $k \ge D$ 时 $V_k = V^*$，再迭代一轮得到 $\delta_{D+1} = 0$，算法判定收敛并返回精确的最优价值。这就是实验中迭代轮数约为 $D+1$、且最后一次更新量恰为 0 的原因（见第 4 节）。对于一般的不确定 MDP，这一有限步精确收敛的性质不再成立，价值迭代只能给出 ε 意义下的近似解。

3.4 策略提取

收敛后，对每个非终点状态 s 按下式提取贪心动作：

$$\pi^*(s) = \arg\max_{a \in \mathcal{A}} \Big\{ R(s,a) + \gamma V\big(f(s,a)\big) \Big\}.$$

终点为吸收态，不定义动作，故策略表中只包含 $|\mathcal{S}| - 1$ 个状态。

贪心策略的最优性来自策略改进定理：由 $\mathcal{T}$ 的不动点性质，对 V^* 取贪心得到的策略满足 $V^{\pi^*} = V^*$。若价值只是近似收敛，记 $\|V_k - V^*\|_\infty \le \varepsilon$，则可以证明贪心策略满足

$$\|V^{\pi_k} - V^*\|_\infty \le \frac{2\gamma\varepsilon}{1-\gamma},$$

即策略的次优程度随价值误差线性下降；在本题的精确收敛情形下，该误差为零，返回的策略就是最优策略。

当多个动作的 Q 值相同（例如在开放区域中多个方向都能以相同步数到达终点）时，$\arg\max$ 的结果取决于动作的枚举顺序。本实验采用第 2 节定义的固定顺序，即北、东、南、西

$$(-1,0),\quad (0,1),\quad (1,0),\quad (0,-1),$$

并规定在取值相同时选取顺序最靠前的动作，从而使策略唯一、可复现，也保证了策略图在多次运行中保持一致。

3.5 复杂度分析

算法每一轮对 $|\mathcal{S}|$ 个状态、每个状态 $|\mathcal{A}| = 4$ 个动作各做一次常数时间的备份，因此单轮时间复杂度为 $O(|\mathcal{S}||\mathcal{A}|)$。保持整个价值表与策略表各需 $O(|\mathcal{S}|)$ 空间，转移函数通过格点坐标即时计算，不需要预先存储 $|\mathcal{S}| \times |\mathcal{A}|$ 的转移表。

总时间取决于迭代轮数 K：一般情形下由先验界给出 $K = O\left(\dfrac{\log(\varepsilon(1-\gamma)/\gamma)}{\log\gamma}\right)$，在 $\gamma = 0.99$、$\varepsilon = 10^{-8}$ 时约为 2.3×10^3 轮；而在本题中由于有限步精确收敛，$K \le D+1 \le |\mathcal{S}|$，实测轮数为 $2 \sim 47$ 轮。加上一次长度不超过 $D+1$ 的策略提取与轨迹展开，总体复杂度仍为 $O(|\mathcal{S}||\mathcal{A}|)$ 量级，实验中最大规模的 15×15 迷宫（102 个状态）单次求解耗时不足 5 ms。

3.6 实现要点

算法在 `maze.py` 与 `experiment.py` 中实现，核心迭代部分如代码 2 所示。实现上的主要考虑如下：

- **状态与动作编码**：状态用 (行, 列) 元组表示，只枚举非墙体格子，因此价值表与策略表都是 $O(|\mathcal{S}|)$ 的字典，墙体不占用任何条目；动作直接用行列位移表示，与第 2 节的 $\mathcal{A}$ 一致。
- **转移与奖励**：转移函数 f 与奖励 R 合并实现在 `Maze.step` 中，一次调用返回（后继状态, 奖励, 是否终止）三元组。若目标格是墙体或越界，后继取原状态且奖励仍为 -1，从而避免把撞墙当作可以无限重复的“免费动作”。
- **终点处理**：`step` 在终点直接返回 $(s_g, 0, \text{True})$，备份时对终止情形取 0 作为后续价值，与代码 1 中 $V(s_g) = 0$ 的写法完全对应。
- **同步更新与误差记录**：每轮先把结果写入新字典再整体替换，同时记录 δ_k，既保证了与压缩映射理论的一致性，也为第 4 节的收敛曲线提供了数据。
- **平局与可复现性**：策略提取使用带固定顺序的 `max`（Python 的 `max` 在并列时返回第一个元素），使平局时的动作选择确定，便于复现实验与比较不同迷宫的结果。
- **数值安全**：$\gamma < 1$ 保证价值有界，迭代不会发散；同时以 $\delta_k < \varepsilon$ 作为唯一停止条件，避免使用固定轮数而带来的精度不确定性。

代码 2: 核心迭代实现（摘自 `maze.py`）

```
values = dict.fromkeys(maze.states, 0.0)   # V_0 = 0
deltas = []
while True:
    updated = {                            # 同步备份
        state: max(action_value(state, move) for move in MOVES)
        for state in maze.states
    }
    deltas.append(max(abs(updated[s] - values[s])
                      for s in maze.states))
    values = updated
    if deltas[-1] < tolerance:
        break
```

4 实验结果

5 总结